

SOFTWARE ENGINEERING

LAB 3

Component Modelling and Architectural Pattern Selection

OpenAPI Mock Server Generator

Arin Singh

SRN: PES1UG24CS079

1. Objective

In this lab, I studied three architecture styles: Layered Architecture, Microservices Architecture and Client-Server Architecture. I compared them for my assigned system, selected the one that fits best, identified the main software components and made a UML component diagram showing interfaces and dependencies.

2. Problem Statement

My assigned system is an **OpenAPI Mock Server Generator**. The tool accepts an OpenAPI 3.0 file in YAML or JSON format. It checks whether the file is valid and creates mock REST endpoints for every path and HTTP method in the file. When an endpoint is called, it returns an example response or creates a random JSON response that follows the schema. The user can also set an artificial response delay.

The main users are an API Developer and a QA Engineer. An API client or frontend application calls the generated endpoints while the mock server is running.

3. Work Carried Forward from Lab 1 and Lab 2

Lab 1 defined the requirements and the main use case. Lab 2 converted the work into epics, user stories and sprint tasks. I used the same requirements while deciding the architecture in this lab.

Requirement	Requirement used in this lab
FR-001	Accept OpenAPI 3.0 specifications in YAML or JSON format.
FR-002	Validate paths, parameters and schemas, and display useful error messages.
FR-003	Create a mock route for every path and HTTP method in the specification.
FR-004	Return examples or generate schema-compliant JSON responses.
FR-005	Allow the user to configure simulated response latency.
NFR-001	Handle at least 1,000 requests per second and keep the delay within ± 10 ms of the configured value.
NFR-002	Reject malformed or harmful files without crashing, running file content or showing internal information.

Lab information

The suggested duration is 60 minutes. A UML tool such as draw.io, Lucidchart or StarUML can be used. The required outputs are a component diagram and a one-page architecture justification.

4. Scenario Review

4.1 Functional requirements

ID	What the system must do	Effect on the design
FR-001	Take YAML or JSON input.	A simple CLI or local interface is needed to receive the file and settings.
FR-002	Validate the OpenAPI file.	The file must be checked before any route is added to the running server.
FR-003	Create all specified mock routes.	The validated model must be converted into a route table for the HTTP runtime.
FR-004	Create valid response bodies.	Response generation should be separate from basic HTTP request handling.
FR-005	Apply a configured delay.	The delay should use a non-blocking timer so other requests are not stopped.

4.2 Non-functional requirements

Area	Target	Design idea
Performance	At least 1,000 requests per second and delay accuracy within ± 10 ms.	Keep the route model in memory, compile routes once and use asynchronous I/O.
Security and reliability	Unsafe input must not crash the tool, run code or leak internal details.	Treat the file only as data, limit file size and nesting, restrict references and return safe errors.

4.3 Main challenges

- **Performance:** Route matching and response generation should stay fast even when many requests arrive.
- **Usability:** The user should be able to start the mock server easily and understand validation errors.
- **Security:** OpenAPI files are untrusted input, so they must never be executed as code.
- **Reliability:** Invalid files or an already-used port should fail cleanly without leaving partial routes.

Scope decision

A permanent database is not required by Problem Statement #41. The validated API model and route table can stay in memory for the current mock-server session. This keeps the tool smaller and easier to run.

5. Comparison of the Three Architecture Types

5.1 Layered Architecture

Structure: The program is divided into presentation, application, domain and infrastructure layers.

Advantages: It gives a clear place for input, validation, response generation and HTTP handling. The components can be tested separately, while calls can still remain inside one process.

Limitations: Too many layer calls can add overhead. The manager can also become too large if responsibilities are not kept separate.

Fit for this system: High. The steps of the mock server already follow a clear input → validate → configure → serve flow.

5.2 Microservices Architecture

Structure: Parsing, route management, response generation and runtime handling would run as separate services over a network.

Advantages: Each service can be deployed or scaled separately. A failure in one service may be isolated from the others.

Limitations: It requires service discovery, network calls, serialization, retries and extra monitoring. That is a lot of work for a local developer tool and can also make the latency target less predictable.

Fit for this system: Low. The problem does not ask for separate deployment or distributed scaling.

5.3 Client-Server Architecture

Structure: A central mock server provides generated endpoints to one or more API clients.

Advantages: Clients are simple and all endpoint configuration stays in one place.

Limitations: The server can become a bottleneck or single point of failure. This style explains how clients connect to the server, but it does not clearly divide the work inside the server.

Fit for this system: Medium. It is useful as the external communication model, but a separate internal architecture is still needed.

5.4 Direct comparison

Point	Layered	Microservices	Client-Server
Match with workflow	Strong	Possible, but unnecessary	Only explains external communication
Performance	Fast in-process calls	Network overhead	Simple, but central server may bottleneck
Deployment effort	Low	High	Medium
Overall fit	High	Low	Medium

6. Architecture Selection

Selected architecture: Layered Architecture

I selected Layered Architecture because it matches the order in which the tool works. The file is received, validated, converted into routes and then served through the HTTP runtime. Keeping these steps in separate layers makes the code easier to understand and test.

I did not choose Microservices because this tool does not need independent deployment of many services. Client-Server is still used when an external application calls the generated mock endpoints, but it is not enough to organise the internal code.

6.1 Layer arrangement

Layer	Component placed in the layer
Presentation layer	Developer Interface / CLI
Application layer	Mock Server Manager
Domain services	OpenAPI Processor and Response & Latency Engine
Infrastructure layer	Mock Runtime Engine

7. Component Identification

The diagram uses five components. Each component has one main responsibility so that the design is easy to follow.

No.	Component	Responsibility	Requirements
1	Developer Interface / CLI	Receives the OpenAPI file or URL, port and delay settings. It also displays errors and server status.	FR-001, FR-005
2	Mock Server Manager	Coordinates validation, route setup and the start/stop lifecycle of the server.	FR-002, FR-003
3	OpenAPI Processor	Parses YAML/JSON, checks permitted references, validates the specification and returns a normalized API model.	FR-001, FR-002, NFR-002
4	Response & Latency Engine	Uses an example or creates schema-compliant JSON, and applies the configured response delay.	FR-004, FR-005, NFR-001
5	Mock Runtime Engine	Binds the HTTP port, installs the route table, matches requests and sends HTTP responses.	FR-003, NFR-001

Boundary followed: The CLI does not control network sockets directly, and the runtime does not parse the raw OpenAPI file. The Manager coordinates these operations.

8. Interfaces Between Components

ID	Required by	Provided by	Information passed	Form
I-01	Developer Interface / CLI	Mock Server Manager	Specification location, port, delay, start/stop command, status and errors.	CLI or local API
I-02	Mock Server Manager	OpenAPI Processor	Raw YAML/JSON and parsing rules; returns a validated model or safe errors.	Parser/validator API
I-03	Mock Server Manager	Mock Runtime Engine	Compiled route table, port configuration and lifecycle commands.	Lifecycle service
I-04	Mock Runtime Engine	Response & Latency Engine	Matched operation and request values; returns status, headers, JSON body and delay.	Request handler API

8.1 UML notation used

- A component is shown as a rectangle with the <<component>> label.
- A small square on the component boundary is a port.
- A provided interface is shown using ball notation.
- A required interface is shown using socket notation.
- A solid assembly connector joins a required interface to a provided interface.
- A dashed arrow marked <<use>> shows that one component depends on another.

8.2 Data flow during startup

1. The API Developer gives the OpenAPI file, port and delay through I-01.
2. The Manager sends the file to the OpenAPI Processor through I-02.
3. If validation fails, the user receives a safe error and no route is installed.
4. If validation succeeds, the Manager compiles the route configuration.
5. The Manager sends the routes to the Runtime through I-03 and starts the listener.

8.3 Data flow for a request

1. An API client calls one of the generated paths.
2. The Runtime matches the request with the compiled route.
3. The Runtime calls I-04 to obtain a valid response and the configured delay.
4. The delay is applied without blocking other requests.
5. The Runtime sends the HTTP status, headers and JSON body to the client.

The HTTP call from an external client is shown as context and is not counted as one of the four internal component interfaces.

9. UML Component Diagram

The diagram has five components and four internal interfaces. Ports, provided and required interfaces, assembly connectors and <<use>> dependencies are shown.

OpenAPI Mock Server Generator - Layered Architecture

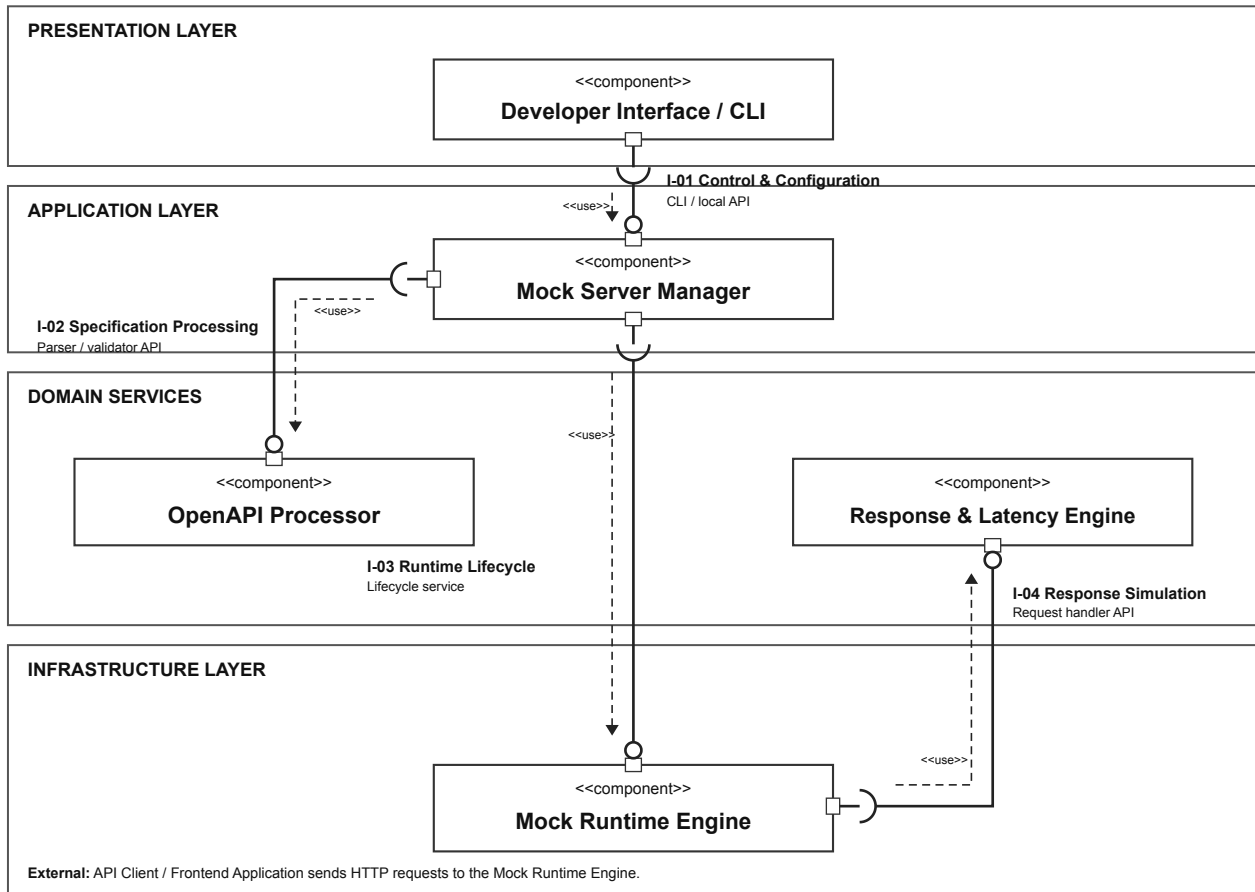


Figure 1. UML Component Diagram for the OpenAPI Mock Server Generator.

10. Requirement Traceability and Work Done

Requirement	Where it is handled	How it can be checked
FR-001	Developer Interface and OpenAPI Processor through I-01 and I-02.	Load equivalent YAML and JSON files and compare the normalized model.
FR-002	OpenAPI Processor validation step.	Use an invalid file and confirm that no route is installed.
FR-003	Manager compiles routes and Runtime installs them through I-03.	Call every defined path/method and also test an undefined path.
FR-004	Response & Latency Engine through I-04.	Check that required fields and JSON types match the selected response schema.
FR-005	Delay setting through I-01 and scheduler through I-04.	Measure the observed delay for different configured values.
NFR-001	Precompiled in-memory routes, asynchronous request handling and non-blocking delay.	Run a load test at 1,000 or more requests per second and measure delay error.
NFR-002	Bounded parsing, permitted references, safe errors and fail-closed setup.	Try malformed and oversized files and confirm that the process does not crash or show stack traces.

Work completed in this lab

- Reviewed the assigned OpenAPI Mock Server Generator scenario.
- Listed the important functional and non-functional requirements.
- Compared Layered, Microservices and Client-Server architectures.
- Selected Layered Architecture and explained the choice.
- Identified five components and their responsibilities.
- Defined four internal interfaces and described the data flow.
- Used provided interfaces, required interfaces, ports, assembly connectors and <<use>> dependencies.
- Prepared the UML component diagram.
- Prepared a one-page justification with two reasons, a security advantage and a performance benefit.

Deliverable check

- At least five components are present.
- At least four interfaces are labelled.
- Interface communication forms are stated.
- Component interaction and data flow are explained.
- The justification is kept to one page.

11. One-Page Architectural Justification

We chose Layered Architecture for the OpenAPI Mock Server Generator.

Architectural Choice

After comparing all three styles, I selected Layered Architecture. The system is divided into a Presentation layer, an Application layer, Domain Services and an Infrastructure layer. The five components communicate through four clear interfaces.

Reason 1 - It matches the working steps

The tool works in a fixed order. It receives an OpenAPI file, checks it, creates a route configuration, starts the HTTP server and then produces mock responses. These steps fit naturally into layers. It also becomes easier to test the parser, response generator and runtime separately.

Reason 2 - It is easier to run and maintain

This is mainly a developer tool, not a large distributed system. Running several microservices would add network setup, service discovery and extra debugging. A single layered application is easier to install and use. Client-Server is still used for calls from external API clients, but Layered Architecture gives a better internal structure.

Security Advantage

The OpenAPI Processor acts as a validation boundary. YAML or JSON input is treated only as data and is checked before the Manager installs any route. File size and nesting can be limited, unsafe references can be rejected, and errors can be shown without internal stack details. The Runtime never reads the raw uploaded file.

Performance Benefit

The components call each other inside the same process, so there is no network serialization between them. Routes and validators can be prepared once and kept in memory. Asynchronous I/O and non-blocking timers help the tool handle at least 1,000 requests per second while keeping the configured delay within ± 10 ms.

Conclusion

Layered Architecture is the best choice for this problem because it is simple, clear and suitable for the required performance. It keeps validation separate from runtime handling and gives a direct implementation plan for the requirements prepared in Lab 1 and the stories planned in Lab 2.